Home Credit Default Risk — Modeling

AUTHOR
Jake Hoopes

1 Project Goal

Home Credit lends to people who often have little or no conventional credit history. The business problem is to estimate the probability that a loan applicant will default, so Home Credit can assess applicant risk more accurately while still extending credit to customers who are able to repay. A model that ranks applicants well lets the lender focus scrutiny on the riskiest applications without turning away qualified borrowers.

2 Analytic Problem

The outcome is the binary variable `TARGET`:

- `TARGET = 1`: the applicant experienced payment difficulties (default).
- `TARGET = 0`: the applicant did not.

The goal is to predict a probability of default for each applicant and, through those probabilities, to rank applicants from most to least risky.

The classes are highly imbalanced: defaults are a small minority. A model can therefore score very high accuracy simply by predicting that nobody defaults, so accuracy alone is not an appropriate model-selection metric. ROC AUC is the main metric in this notebook. It measures how well a model ranks defaulters above non-defaulters across every possible threshold, which is what both the Kaggle competition and this assignment evaluate.

3 Purpose of This Notebook

This notebook will:

1. establish a simple benchmark,
2. compare several candidate models using cross-validation,
3. test class-imbalance strategies,
4. tune the best model,
5. evaluate the selected model once on an untouched internal holdout set,
6. train the selected final model on all labeled data,
7. generate Kaggle predictions, and
8. summarize the results, findings, and limitations.

4 Modeling Data

The modeling tables come from `data_preparation.R` at the repository root, which applies identical, train-learned transformations to `application_train` and `application_test`. This notebook reads its saved outputs rather than repeating the preparation.

```
train_prepared <- readRDS("data/prepared/train_prepared.rds")
test_prepared  <- readRDS("data/prepared/test_prepared.rds")

# Outcome, and predictors with the ID and target removed. SK_ID_CURR is an
# identifier and is never used as a predictor.
y <- train_prepared$TARGET

x_train <- train_prepared |> select(-SK_ID_CURR, -TARGET)
x_test  <- test_prepared  |> select(-SK_ID_CURR)

# Kept separately to build the Kaggle submission later.
test_ids <- test_prepared$SK_ID_CURR
```

The summary below confirms the two tables line up before any modeling begins.

```
tibble(
  item = c("Training rows",
           "Test rows",
           "Predictor columns (train)",
           "Predictor columns (test)",
           "Predictors identical in train and test",
           "TARGET only in train",
           "SK_ID_CURR in both",
           "Missing values in train",
           "Missing values in test"),
  # format() each entry separately so logicals print as TRUE/FALSE, not 1/0.
  value = c(format(nrow(x_train), big.mark = ","),
            format(nrow(x_test), big.mark = ","),
            format(ncol(x_train)),
            format(ncol(x_test)),
            format(identical(names(x_train), names(x_test))),
            format("TARGET" %in% names(train_prepared) && !"TARGET" %in%
         names(test_prepared)),
            format("SK_ID_CURR" %in% names(train_prepared) && "SK_ID_CURR" %in%
         names(test_prepared)),
            format(sum(is.na(train_prepared))),
            format(sum(is.na(test_prepared))))
) |>
  knitr::kable(caption = "Modeling data summary")
```

Modeling data summary

item	value
Training rows	307,511

item	value
Test rows	48,744
Predictor columns (train)	167
Predictor columns (test)	167
Predictors identical in train and test	TRUE
TARGET only in train	TRUE
SK_ID_CURR in both	TRUE
Missing values in train	0
Missing values in test	0

Categorical predictors remain factors at this stage. Any dummy encoding needed by a particular model will be done later, inside the modeling step that needs it.

4.1 Data Preparation Used for Modeling

The 167 predictors were built by `data_preparation.R`, which implements the decisions recorded in the EDA notebook. In brief:

- **Cleaning.** The `DAYS_EMPLOYED` placeholder value (365243) is recoded to missing and replaced by a `not_currently_employed` flag. Income is capped at the training 99th percentile. The six credit-bureau inquiry counts are replaced by a single `no_inquiry_data` flag, and the constant `FLAG_MOBIL` is dropped.
- **Engineered features.** These include age and tenure in years, affordability ratios (loan-to-value, payment-to-income, credit-to-income, credit-to-annuity, income per person), summaries of the three external credit scores (mean, minimum, maximum, count), a count of blank fields, binned versions of nonlinear relationships, and three interaction terms.
- **Bureau data.** `bureau.csv` is aggregated to one row per applicant (number of credits, active credits and debt, overdue amounts, recency, and related ratios), with a `has_bureau_record` flag for applicants who have no bureau history.
- **Missing values.** No rows are dropped. Numeric gaps are filled with training medians, 18 `_missing` indicator flags keep the fact that a value was absent where that mattered for default, and missing categories become an explicit `"Missing"` level. Rare categorical levels are pooled.
- **Train/test consistency.** Every learned value (income cap, medians, cut points, factor levels, and the choice of flag columns) is learned from `application_train` only and applied unchanged to `application_test`, so both files receive identical transformations.

In this notebook, factors are dummy-encoded by caret's formula interface at fit time. For logistic regression, predictors are also centered and scaled inside each training fold.

5 Target Distribution

```
target_table <- tibble(TARGET = y) |>
  count(TARGET, name = "count") |>
  mutate(percent = 100 * count / sum(count))
```

```r
default_rate  <- mean(y == 1)
majority_rate <- max(target_table$count) / length(y)

target_table |>
  knitr::kable(digits = 2, format.args = list(big.mark = ","), caption = "TARGET
        distribution in the training data")
```

TARGET distribution in the training data

TARGET	count	percent
0	282,686	91.93
1	24,825	8.07

Only 8.07% of training applicants defaulted, so the majority class (`TARGET = 0`) makes up 91.93% of the data. That means there are roughly 11.4 non-defaulters for every defaulter. Two consequences follow. First, any accuracy figure has to be judged against this majority share rather than against 50%. Second, an unadjusted model sees few positive examples, which is why later sections compare models with and without class-imbalance adjustments.

6 Performance Benchmark

The simplest possible classifier predicts the majority class, `TARGET = 0`, for every applicant. Its hard predictions give its accuracy. For AUC, the same model assigns every applicant an identical predicted probability (the training default rate), so it cannot rank anyone above anyone else.

```r
# Hard predictions: everyone is classed as a non-defaulter.
benchmark_class <- rep(0, length(y))
benchmark_accuracy <- mean(benchmark_class == y)

# Probability scores: a constant for every applicant, so no ranking exists.
benchmark_prob <- rep(default_rate, length(y))
benchmark_auc <- as.numeric(auc(roc(y, benchmark_prob, levels = c(0, 1),
                                    direction = "<", quiet = TRUE)))

results <- tibble(
  Model      = "Majority Class",
  Accuracy   = benchmark_accuracy,
  `ROC AUC`  = benchmark_auc
)

results |>
  knitr::kable(digits = 4, caption = "Benchmark performance on the training data")
```

Benchmark performance on the training data

Model	Accuracy	ROC AUC
Majority Class	0.9193	0.5

The benchmark reaches 91.93% accuracy without using a single predictor. It is correct for every non-defaulter, and because `TARGET = 0` dominates the data, that alone produces a high score. The accuracy is misleading: the model identifies none of the defaulters, which are exactly the applicants the business problem is about.

Its ROC AUC is 0.50. Because every applicant receives the same score, the model cannot place a defaulter above a non-defaulter more often than chance, so it has no discrimination or ranking ability. An AUC of 0.50 is therefore the floor a useful model must clear. Candidate models in the following sections will be compared primarily on ROC AUC, with accuracy reported only for context.

7 Validation Considerations

The prepared data were created before any cross-validation. Most of the parameters learned during preparation (imputation medians, factor levels, the income cap, and the payment-to-income cut points) did not use `TARGET`, so learning them on the full training set raises limited concern.

However, some missing-value indicators were selected by comparing default rates across the complete training set, and several feature choices, such as the fixed bands, were read from EDA plots of the default rate on that same data. Because this `TARGET` information was used before the data were split into folds, cross-validation estimates may be slightly optimistic. The preparation pipeline is not being rebuilt at this stage. This is a known limitation of the current workflow and should be kept in mind when interpreting the CV results that follow.

The following rules hold throughout the rest of the notebook:

- The Kaggle test set is never used for model selection or tuning; it is used only to generate final predictions.
- `SK_ID_CURR` is excluded from the predictors.
- Any class balancing (weights or resampling) is applied only within the training portion of each CV fold, never to the full data before splitting.

8 Internal Validation Strategy

`application_test` has no `TARGET` labels, so it cannot be used to estimate model performance. Instead, the labeled training data are split once, reproducibly, into two parts:

- **Development data (about 80%)**, used for every model comparison, imbalance experiment, and tuning step from here on.
- **Holdout data (about 20%)**, set aside and used only once, at the end, to evaluate the selected model.

The split is stratified on `TARGET`, so both parts keep the same default rate. The holdout set is never used for model selection, feature selection, hyperparameter tuning, choosing an imbalance strategy, or choosing a classification threshold. `SK_ID_CURR` stays in both data frames for identification only and is never a predictor.

```r
set.seed(6493)
dev_index <- createDataPartition(factor(train_prepared$TARGET), p = 0.8,
                                 list = FALSE)[, 1]

development_data <- train_prepared[dev_index, ]
holdout_data     <- train_prepared[-dev_index, ]

split_summary <- function(df, label) {
  tibble(Dataset        = label,
         Rows           = nrow(df),
         `TARGET = 0`   = sum(df$TARGET == 0),
         `TARGET = 1`   = sum(df$TARGET == 1),
         `Default Rate` = scales::percent(mean(df$TARGET == 1), accuracy = 0.01))
}

bind_rows(split_summary(train_prepared,   "Full labeled training data"),
          split_summary(development_data, "Development data"),
          split_summary(holdout_data,     "Holdout data")) |>
  knitr::kable(format.args = list(big.mark = ","),
               caption = "Development / holdout split")
```

Development / holdout split

Dataset	Rows	TARGET = 0	TARGET = 1	Default Rate
Full labeled training data	307,511	282,686	24,825	8.07%
Development data	246,009	226,149	19,860	8.07%
Holdout data	61,502	56,537	4,965	8.07%

The default rate is 8.07% in the development data and 8.07% in the holdout data, essentially identical to the full training data, so the stratification worked as intended.

8.1 Exploration Sample

Fitting every candidate model on all 246,009 development rows would be slow, and the assignment emphasizes computational efficiency during model development. The initial comparison therefore uses one stratified random sample of about 50,000 development rows.

```r
set.seed(2026)
explore_index <- createDataPartition(factor(development_data$TARGET),
                                     p = 50000 / nrow(development_data),
                                     list = FALSE)[, 1]
exploration_data <- development_data[explore_index, ]

tibble(Rows           = nrow(exploration_data),
       Defaults       = sum(exploration_data$TARGET == 1),
       `Non-defaults` = sum(exploration_data$TARGET == 0),
```

```
        `Default Rate` = scales::percent(mean(exploration_data$TARGET == 1), accuracy =
         0.01)) |>
  knitr::kable(format.args = list(big.mark = ","), caption = "Exploration sample")
```

Exploration sample

Rows	Defaults	Non-defaults	Default Rate
50,001	4,037	45,964	8.07%

The same exploration sample is used for all three candidate models, so differences in their AUC and runtime reflect the models rather than the data each one happened to see.

8.2 Cross-Validation Design

caret's `twoClassSummary` needs the outcome as a factor with valid R names, and it treats the **first** factor level as the event whose probability is scored. `TARGET` is therefore recoded so that `Default` (`TARGET = 1`) is the first level. A single set of stratified 3-fold CV indexes is created once and passed to every model, so all three are trained and scored on exactly the same rows.

```
# Predictors are every prepared column except SK_ID_CURR; TARGET is the outcome.
model_data <- exploration_data |>
  mutate(TARGET = factor(if_else(TARGET == 1, "Default", "NoDefault"),
                         levels = c("Default", "NoDefault"))) |>
  select(-SK_ID_CURR)

set.seed(6610)
cv_folds <- createFolds(model_data$TARGET, k = 3, returnTrain = TRUE)

# One seed per fold (one tuning combination each) plus one for caret's final
# refit, so results are reproducible even when folds run in parallel.
cv_seeds <- c(map(1:3, ~ sample.int(100000, 1)), list(sample.int(100000, 1)))

cv_control <- trainControl(
  method          = "cv",
  number          = 3,
  index           = cv_folds,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final",
  seeds           = cv_seeds,
  allowParallel   = TRUE
)

tibble(Fold                 = names(cv_folds),
       `Training rows`      = map_int(cv_folds, length),
       `Assessment rows`    = map_int(cv_folds, ~ nrow(model_data) - length(.x)),
       `Assessment defaults` = map_dbl(cv_folds, ~ mean(model_data$TARGET[-.x] ==
        "Default")) |>
         scales::percent(accuracy = 0.01)) |>
  knitr::kable(format.args = list(big.mark = ","),
```

```
                caption = paste0("Shared 3-fold CV (event level: ",
                                 levels(model_data$TARGET)[1], ")"))
```

Shared 3-fold CV (event level: Default)

Fold	Training rows	Assessment rows	Assessment defaults
Fold1	33,334	16,667	8.08%
Fold2	33,333	16,668	8.08%
Fold3	33,335	16,666	8.07%

Every model is selected on `metric = "ROC"`, the cross-validated ROC AUC. Accuracy is not used for model selection.

9 Candidate Models

All three models below are **unadjusted**: no class weights, resampling, or other imbalance correction is applied. They use the prepared predictors as they are, through caret's formula interface, which turns factors into dummy columns for the algorithms that need numeric input. No new predictors are built and no categories are target-encoded.

```
# Number of predictor columns after caret's dummy encoding, used to set mtry.
n_model_predictors <- ncol(model.matrix(TARGET ~ ., data = model_data)) - 1

# Three folds run in parallel, one worker each. Every model is kept
# single-threaded inside its worker to avoid oversubscribing the CPU.
cl <- makePSOCKcluster(3)
registerDoParallel(cl)
```

After dummy encoding there are 300 predictor columns. The three folds run in parallel on 3 workers, and each model runs single-threaded inside its worker. Runtimes below are elapsed (wall-clock) times and include caret's final refit on the full exploration sample.

9.1 Regularized Logistic Regression

Logistic regression is a relatively interpretable linear benchmark: each predictor's effect is a single coefficient on the log-odds of default. This comparison uses one fixed ridge configuration (`alpha = 0`, `lambda = 0.01`), which shrinks coefficients to cope with the many correlated predictors without dropping any. Predictors are centered and scaled inside each training fold, and dummy columns with zero variance in a fold are dropped, because glmnet's penalty depends on the scale of each predictor.

```
time_glmnet <- system.time(
  fit_glmnet <- train(
    TARGET ~ ., data = model_data,
    method     = "glmnet",
    metric     = "ROC",
    trControl  = cv_control,
```

```
    preProcess = c("zv", "center", "scale"),
    tuneGrid   = data.frame(alpha = 0, lambda = 0.01)
  )
)["elapsed"]
```

9.2 Random Forest

A random forest averages many decision trees, each grown on a bootstrap sample with a random subset of predictors considered at each split. It can capture nonlinear relationships and interactions without their being specified. To keep runtime modest this comparison uses 200 trees and one `mtry` value, the usual classification default of the square root of the number of predictors (17).

```
time_rf <- system.time(
  fit_rf <- train(
    TARGET ~ ., data = model_data,
    method    = "rf",
    metric    = "ROC",
    trControl = cv_control,
    tuneGrid  = data.frame(mtry = floor(sqrt(n_model_predictors))),
    ntree     = 200
  )
)["elapsed"]
```

9.3 XGBoost

Gradient-boosted trees are built sequentially, each new shallow tree fitted to correct the errors of the trees before it. They often perform strongly on structured, tabular data like this. This comparison uses one fixed, conventional starting configuration:

Parameter	Value	Role
`nrounds`	250	number of boosting rounds
`max_depth`	5	maximum tree depth
`eta`	0.05	learning rate
`gamma`	0	minimum loss reduction to split
`colsample_bytree`	0.8	share of predictors sampled per tree
`min_child_weight`	1	minimum leaf weight
`subsample`	0.8	share of rows sampled per tree

caret 7.0.1's built-in `xgbTree` method does not work with the installed xgboost 3.x: caret tries to attach metadata to the fitted booster, which xgboost 3.x no longer allows, so every fold fails. The chunk below copies caret's `xgbTree` definition and changes only how the model is stored and called (the booster is wrapped in a list, and the objective and thread count are passed as xgboost 3.x expects). Like caret's original, it passes any observation weights supplied to `train()` through to XGBoost; the class-imbalance section relies on this. The algorithm, parameters, folds, and caret workflow are unchanged.

```
xgb_tree_model <- getModelInfo("xgbTree", regex = FALSE)$xgbTree

# Single parameter combination, so caret's sub-model shortcut is not needed.
xgb_tree_model$loop <- NULL

xgb_tree_model$fit <- function(x, y, wts, param, lev, last, classProbs, ...) {
  dots <- list(...)
  # The first factor level (Default) is the event, coded 1.
  label <- as.numeric(y == lev[1])
  booster <- xgboost::xgb.train(
    params = list(objective        = "binary:logistic",
                  eval_metric      = "logloss",
                  eta              = param$eta,
                  max_depth        = param$max_depth,
                  gamma            = param$gamma,
                  colsample_bytree = param$colsample_bytree,
                  min_child_weight = param$min_child_weight,
                  subsample        = param$subsample,
                  nthread          = if (is.null(dots$nthread)) 1 else dots$nthread),
    # Observation weights, as in caret's original xgbTree. caret subsets them
    # to each training fold; wts is NULL when no weights are supplied.
    data    = xgboost::xgb.DMatrix(as.matrix(x), label = label, weight = wts),
    nrounds = param$nrounds,
    verbose = 0
  )
  list(booster = booster)
}

xgb_tree_model$prob <- function(modelFit, newdata, submodels = NULL) {
  p <- predict(modelFit$booster, as.matrix(newdata))
  out <- data.frame(p, 1 - p)
  names(out) <- modelFit$obsLevels
  out
}

xgb_tree_model$predict <- function(modelFit, newdata, submodels = NULL) {
  p <- predict(modelFit$booster, as.matrix(newdata))
  ifelse(p >= 0.5, modelFit$obsLevels[1], modelFit$obsLevels[2])
}

xgb_tree_model$varImp <- NULL
```

```
time_xgb <- system.time(
  fit_xgb <- train(
    TARGET ~ ., data = model_data,
    method    = xgb_tree_model,
    metric    = "ROC",
    trControl = cv_control,
    tuneGrid  = data.frame(nrounds = 250, max_depth = 5, eta = 0.05,
                           gamma = 0, colsample_bytree = 0.8,
```

```
                          min_child_weight = 1, subsample = 0.8),
    nthread   = 1
  )
)["elapsed"]
```

```
stopCluster(cl)
registerDoSEQ()
```

10 Initial Model Comparison

```
cv_fits <- list(
  "Regularized Logistic Regression" = fit_glmnet,
  "Random Forest"                   = fit_rf,
  "XGBoost"                         = fit_xgb
)
runtimes_min <- c(time_glmnet, time_rf, time_xgb) / 60

cv_results <- imap(cv_fits, function(fit, model) {
  fit$resample |>
    select(Resample, ROC) |>
    mutate(Model = model)
}) |>
  list_rbind() |>
  group_by(Model) |>
  mutate(`Mean CV AUC` = mean(ROC), `SD CV AUC` = sd(ROC)) |>
  ungroup() |>
  mutate(Resample = paste(str_replace(Resample, "Fold", "Fold "), "AUC")) |>
  pivot_wider(names_from = Resample, values_from = ROC) |>
  left_join(tibble(Model = names(cv_fits), `Runtime (min)` = runtimes_min),
            by = "Model") |>
  select(Model, `Fold 1 AUC`, `Fold 2 AUC`, `Fold 3 AUC`,
         `Mean CV AUC`, `SD CV AUC`, `Runtime (min)`) |>
  arrange(desc(`Mean CV AUC`))

cv_results |>
  knitr::kable(digits = c(0, 4, 4, 4, 4, 4, 2),
               caption = "Unadjusted candidate models: shared 3-fold CV on the
        exploration sample")
```

Unadjusted candidate models: shared 3-fold CV on the exploration sample

Model	Fold 1 AUC	Fold 2 AUC	Fold 3 AUC	Mean CV AUC	SD CV AUC	Runtime (min)
XGBoost	0.7551	0.7542	0.7633	0.7575	0.0050	0.17
Regularized Logistic Regression	0.7434	0.7459	0.7526	0.7473	0.0047	0.64
Random Forest	0.7265	0.7256	0.7265	0.7262	0.0005	6.49

XGBoost has the highest mean CV AUC, 0.7575, an improvement of 0.2575 over the 0.50 majority-class benchmark. Regularized logistic regression follows at 0.7473, and random forest is lowest at 0.7262. All

three models rank applicants far better than the benchmark does.

The fold-to-fold results are stable. XGBoost and logistic regression each vary by about 0.005 (standard deviation) across folds, and random forest varies even less. Because all three models were scored on the same folds, they can be compared fold by fold. XGBoost beats logistic regression in every fold, by 0.0083 to 0.0117, so its lead is consistent rather than the product of one favorable split. Both of those models score highest on Fold 3, which suggests that fold is simply somewhat easier to rank.

Runtimes differ sharply. XGBoost was the fastest at 0.17 minutes, logistic regression took 0.64 minutes, and random forest took 6.49 minutes, about 39 times as long as XGBoost.

Added complexity does not automatically improve discrimination here. The boosted model improves on the linear model, but random forest, the most expensive model to fit, scored below plain ridge logistic regression. These results come from a single, untuned configuration of each model on the 50,001-row exploration sample, so they rank the candidates rather than settle their final performance. They identify XGBoost as the strongest candidate so far. It is not yet the final model: the next step tests it with class-imbalance strategies.

11 Addressing Class Imbalance

Only 8.07% of training applicants have `TARGET = 1`. The majority-class benchmark showed what that means for accuracy: predicting “no default” for everyone is right 91.93% of the time while identifying no defaulters. A model trained on data this imbalanced sees about 11.4 non-defaulters for every defaulter, and an imbalance adjustment may help it learn the minority class.

This section tests whether that helps XGBoost **rank** applicants, by comparing three training strategies:

1. **No adjustment**: the XGBoost model from the initial comparison.
2. **Positive-class weighting**: each default counts more heavily in the training loss.
3. **Random downsampling**: non-defaults are randomly discarded from each training fold until the classes are balanced.

Everything else is held fixed: the same exploration sample, predictors, positive class, XGBoost parameters, and the same three CV folds. The only difference is how the imbalance is handled. SMOTE is not used, and the holdout set is not touched.

11.1 Positive-Class Weighting

Defaults receive a weight equal to the ratio of non-defaults to defaults in the exploration sample, and non-defaults receive weight 1. The weighted defaults then carry the same total weight as the non-defaults. The weights go to `train()`, caret subsets them to each training fold, and the custom XGBoost method passes them to XGBoost as observation weights. For binary XGBoost this is equivalent to `scale_pos_weight`. Assessment folds are scored without weights.

```
n_default    <- sum(model_data$TARGET == "Default")
n_no_default <- sum(model_data$TARGET == "NoDefault")
```

```
positive_weight <- n_no_default / n_default

class_weights <- if_else(model_data$TARGET == "Default", positive_weight, 1)
```

The calculated positive-class weight is 11.3857 (45,964 non-defaults / 4,037 defaults).

```
# Same three folds and workers as the initial comparison.
cl <- makePSOCKcluster(3)
registerDoParallel(cl)

time_xgb_weighted <- system.time(
  fit_xgb_weighted <- train(
    TARGET ~ ., data = model_data,
    weights   = class_weights,
    method    = xgb_tree_model,
    metric    = "ROC",
    trControl = cv_control,
    tuneGrid  = fit_xgb$bestTune,
    nthread   = 1
  )
)["elapsed"]
```

11.2 Random Downsampling

With `sampling = "down"`, caret downsamples inside the resampling loop. In each fold it randomly drops non-defaults from the **training** portion until the two classes are equal in size, fits the model, and then scores the untouched assessment fold, which keeps its natural default rate. The full exploration sample is never downsampled before splitting. Apart from `sampling`, the control settings below are identical to `cv_control`, including the fold indexes and seeds.

```
cv_control_down <- trainControl(
  method          = "cv",
  number          = 3,
  index           = cv_folds,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final",
  seeds           = cv_seeds,
  allowParallel   = TRUE,
  sampling        = "down"
)

time_xgb_down <- system.time(
  fit_xgb_down <- train(
    TARGET ~ ., data = model_data,
    method    = xgb_tree_model,
    metric    = "ROC",
    trControl = cv_control_down,
```

```
    tuneGrid  = fit_xgb$bestTune,
    nthread   = 1
  )
)["elapsed"]

stopCluster(cl)
registerDoSEQ()
```

The check below confirms that downsampling touched only the training folds. Each assessment fold's out-of-fold predictions cover every one of its rows at the natural default rate. Each training fold, once downsampled, would hold twice its number of defaults.

```
fit_xgb_down$pred |>
  group_by(Fold = Resample) |>
  summarise(`Assessment rows scored` = n(),
            `Assessment default rate` = scales::percent(mean(obs == "Default"),
                                                        accuracy = 0.01)) |>
  mutate(`Training rows before downsampling` = map_int(cv_folds, length),
         `Training rows after downsampling`  =
           map_int(cv_folds, ~ 2L * sum(model_data$TARGET[.x] == "Default"))) |>
  knitr::kable(format.args = list(big.mark = ","),
               caption = "Downsampling applied to training folds only")
```

Downsampling applied to training folds only

Fold	Assessment rows scored	Assessment default rate	Training rows before downsampling	Training rows after downsampling
Fold1	16,667	8.08%	33,334	5,382
Fold2	16,668	8.08%	33,333	5,382
Fold3	16,666	8.07%	33,335	5,384

11.3 Imbalance Results

The unadjusted row reuses the fold results of the XGBoost model already fitted in the initial comparison, so it was not refit. Its runtime is the time recorded there, not new computation.

```
imbalance_fits <- list(
  "No adjustment"            = fit_xgb,
  "Positive-class weighting" = fit_xgb_weighted,
  "Random downsampling"      = fit_xgb_down
)
imbalance_runtimes <- c(time_xgb, time_xgb_weighted, time_xgb_down) / 60

imbalance_results <- imap(imbalance_fits, function(fit, strategy) {
  fit$resample |>
    select(Resample, ROC) |>
    mutate(Strategy = strategy)
}) |>
```

```
  list_rbind() |>
  group_by(Strategy) |>
  mutate(`Mean CV AUC` = mean(ROC), `SD CV AUC` = sd(ROC)) |>
  ungroup() |>
  mutate(Resample = paste(str_replace(Resample, "Fold", "Fold "), "AUC")) |>
  pivot_wider(names_from = Resample, values_from = ROC) |>
  left_join(tibble(Strategy        = names(imbalance_fits),
                   `Runtime (min)` = imbalance_runtimes,
                   `Fit in this section` = c("No (reused)", "Yes", "Yes")),
            by = "Strategy") |>
  select(Strategy, `Fold 1 AUC`, `Fold 2 AUC`, `Fold 3 AUC`,
         `Mean CV AUC`, `SD CV AUC`, `Runtime (min)`, `Fit in this section`) |>
  arrange(desc(`Mean CV AUC`))

imbalance_results |>
  knitr::kable(digits = c(0, 4, 4, 4, 4, 4, 2, 0),
               caption = "XGBoost imbalance strategies: shared 3-fold CV on the
        exploration sample")
```

XGBoost imbalance strategies: shared 3-fold CV on the exploration sample

Strategy	Fold 1 AUC	Fold 2 AUC	Fold 3 AUC	Mean CV AUC	SD CV AUC	Runtime (min)	Fit in this section
No adjustment	0.7551	0.7542	0.7633	0.7575	0.0050	0.17	No (reused)
Positive-class weighting	0.7436	0.7500	0.7605	0.7514	0.0085	0.23	Yes
Random downsampling	0.7448	0.7500	0.7538	0.7495	0.0045	0.09	Yes

As an independent check, each fold's AUC is recomputed with pROC from the saved out-of-fold probabilities and compared with caret's value.

```
imap(imbalance_fits, function(fit, strategy) {
  fit$pred |>
    group_by(Resample) |>
    summarise(pROC_auc = as.numeric(auc(obs, Default,
                                        levels = c("NoDefault", "Default"),
                                        direction = "<", quiet = TRUE))) |>
    left_join(select(fit$resample, Resample, caret_auc = ROC), by = "Resample") |>
    summarise(Strategy = strategy,
              `Max |pROC - caret|` = max(abs(pROC_auc - caret_auc)))
}) |>
  list_rbind() |>
  knitr::kable(digits = 10, caption = "pROC verification of caret's fold AUCs")
```

pROC verification of caret's fold AUCs

Strategy	Max |pROC - caret|
No adjustment	0
Positive-class weighting	0
Random downsampling	0

11.4 Secondary Diagnostics

Sensitivity (the share of actual defaults flagged) and specificity (the share of non-defaults correctly passed) depend on a classification threshold, so they are **secondary** and are not used to choose a strategy. They are calculated here from the pooled out-of-fold predictions at the same 0.5 threshold for every strategy. The mean predicted default probability, compared with the actual 8.07% rate, shows how each strategy shifts the probability scale.

```
threshold <- 0.5

imbalance_diagnostics <- imap(imbalance_fits, function(fit, strategy) {
  fit$pred |>
    summarise(Strategy                    = strategy,
              Sensitivity                 = mean(Default[obs == "Default"] >=
        threshold),
              Specificity                 = mean(Default[obs == "NoDefault"] <
        threshold),
              `Mean predicted P(default)` = mean(Default))
}) |>
  list_rbind()

imbalance_diagnostics |>
  knitr::kable(digits = 4,
               caption = "Out-of-fold sensitivity and specificity at a 0.5 threshold
        (secondary)")
```

Out-of-fold sensitivity and specificity at a 0.5 threshold (secondary)

Strategy	Sensitivity	Specificity	Mean predicted P(default)
No adjustment	0.0255	0.9980	0.0785
Positive-class weighting	0.5806	0.7892	0.3388
Random downsampling	0.6961	0.6843	0.4121

11.5 Imbalance Interpretation

Neither adjustment improved ranking performance. Positive-class weighting lowered mean CV AUC from 0.7575 to 0.7514 (-0.0062). Random downsampling lowered it to 0.7495 (-0.0080). The unadjusted model produced the strongest ranking performance.

The result is consistent across folds. The unadjusted model has the highest AUC in all three folds, so its lead does not depend on a single split. The fold spread is similar for unadjusted and downsampled XGBoost; weighting has the widest spread (SD 0.0085), with its Fold 1 and Fold 3 scores 0.0169 apart. Downsampling was the fastest of the new runs (0.09 minutes against 0.23 for weighting), because each model was trained on only the balanced subset of each training fold.

The adjustments did change the threshold-dependent diagnostics. At the 0.5 threshold, the unadjusted model flags only 2.6% of defaults. Weighting raises sensitivity to 58.1% and downsampling to 69.6%, while specificity falls from 99.8% to 78.9% and 68.4%. The adjustments also push predicted

probabilities well above the actual default rate. This is the expected effect of rebalancing. It moves predictions toward the default class, which changes what happens at a fixed threshold, but it does not order applicants better. AUC measures that ordering. A higher sensitivity at one threshold therefore does not outweigh the lower AUC. If more defaults need to be flagged, the threshold on the unadjusted model can be lowered instead. No operating threshold is chosen in this project.

The imbalance experiment did not improve ranking performance, so the **unadjusted XGBoost model will proceed to hyperparameter tuning**.

12 Hyperparameter Tuning

The XGBoost configuration used so far was deliberately simple and fixed, so that the candidate models and imbalance strategies could be compared on equal terms. This section searches for better settings with a randomized search on about 5,000 development rows, 3-fold stratified CV, and 20 randomly drawn parameter combinations, optimizing ROC AUC. The smaller sample keeps the 60 model fits cheap while still giving every combination a fair, cross-validated comparison. Tuning uses the unadjusted model, with no class weights or resampling, and no classification threshold is tuned.

12.1 Tuning Sample

The tuning sample is drawn from the development data only. To match the development default rate as closely as possible, the number of defaults is fixed first, the required number of rows is sampled from each class, and the combined rows are shuffled.

```
tuning_n         <- 5000
dev_default_rate <- mean(development_data$TARGET == 1)
tuning_n_default <- round(tuning_n * dev_default_rate)

set.seed(64930)
default_rows    <- sample(which(development_data$TARGET == 1), tuning_n_default)
no_default_rows <- sample(which(development_data$TARGET == 0), tuning_n -
        tuning_n_default)
tuning_data     <- development_data[sample(c(default_rows, no_default_rows)), ]

# Same outcome coding and predictors as before; SK_ID_CURR is dropped.
tuning_model_data <- tuning_data |>
  mutate(TARGET = factor(if_else(TARGET == 1, "Default", "NoDefault"),
                         levels = c("Default", "NoDefault"))) |>
  select(-SK_ID_CURR)

tibble(Rows                          = nrow(tuning_data),
       `TARGET = 0`                  = sum(tuning_data$TARGET == 0),
       `TARGET = 1`                  = sum(tuning_data$TARGET == 1),
       `Default Rate`                = scales::percent(mean(tuning_data$TARGET == 1),
        accuracy = 0.01),
       `Development Default Rate`    = scales::percent(dev_default_rate, accuracy = 0.01),
       `SK_ID_CURR among predictors` = "SK_ID_CURR" %in% names(tuning_model_data),
```

```
        `Same predictors as before`  = identical(names(tuning_model_data),
         names(model_data))) |>
  knitr::kable(format.args = list(big.mark = ","), caption = "Tuning sample")
```

Tuning sample

Rows	TARGET = 0	TARGET = 1	Default Rate	Development Default Rate	SK_ID_CURR among predictors	Same predictors as before
5,000	4,596	404	8.08%	8.07%	FALSE	TRUE

12.2 Tuning Folds and Search Grid

A new set of stratified 3-fold CV indexes is created within the tuning sample, and every parameter combination is scored on these same three folds. The search grid holds 20 combinations drawn at random from the ranges below, not an exhaustive grid. Each parameter is drawn independently, and duplicates are removed before the first 20 are kept.

Parameter	Range searched
`nrounds`	100 to 500, in steps of 50
`max_depth`	2 to 8
`eta`	0.02 to 0.20
`gamma`	0 to 5
`colsample_bytree`	0.60 to 1.00
`min_child_weight`	1 to 10
`subsample`	0.60 to 1.00

```
n_configs <- 20

set.seed(64931)
tuning_folds <- createFolds(tuning_model_data$TARGET, k = 3, returnTrain = TRUE)

set.seed(64932)
xgb_random_grid <- tibble(
  nrounds          = sample(seq(100, 500, by = 50), 2 * n_configs, replace = TRUE),
  max_depth        = sample(2:8, 2 * n_configs, replace = TRUE),
  eta              = round(runif(2 * n_configs, 0.02, 0.20), 3),
  gamma            = round(runif(2 * n_configs, 0, 5), 2),
  colsample_bytree = round(runif(2 * n_configs, 0.60, 1.00), 2),
  min_child_weight = sample(1:10, 2 * n_configs, replace = TRUE),
  subsample        = round(runif(2 * n_configs, 0.60, 1.00), 2)
) |>
  distinct() |>
  slice_head(n = n_configs) |>
  as.data.frame()

# One seed per parameter combination in each fold, plus one for the final refit.
tuning_seeds <- c(map(1:3, ~ sample.int(100000, n_configs)),
                  list(sample.int(100000, 1)))
```

```
tuning_control <- trainControl(
  method          = "cv",
  number          = 3,
  index           = tuning_folds,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final",
  seeds           = tuning_seeds,
  allowParallel   = TRUE
)

tibble(Fold                  = names(tuning_folds),
       `Training rows`       = map_int(tuning_folds, length),
       `Assessment rows`     = map_int(tuning_folds, ~ nrow(tuning_model_data) -
        length(.x)),
       `Assessment defaults` = map_int(tuning_folds, ~ sum(tuning_model_data$TARGET[-.x]
        == "Default"))) |>
  knitr::kable(format.args = list(big.mark = ","),
               caption = paste(nrow(xgb_random_grid),
                               "unique random combinations, scored on these tuning
        folds"))
```

20 unique random combinations, scored on these tuning folds

Fold	Training rows	Assessment rows	Assessment defaults
Fold1	3,334	1,666	134
Fold2	3,333	1,667	135
Fold3	3,333	1,667	135

12.3 Randomized Search

All 20 combinations are fit on the same three folds (60 fits), and caret then refits the best combination on the full tuning sample. The fits are spread across 3 workers, with each XGBoost model single-threaded.

```
cl <- makePSOCKcluster(3)
registerDoParallel(cl)

time_tuning <- system.time(
  fit_xgb_tuned <- train(
    TARGET ~ ., data = tuning_model_data,
    method    = xgb_tree_model,
    metric    = "ROC",
    trControl = tuning_control,
    tuneGrid  = xgb_random_grid,
    nthread   = 1
  )
)["elapsed"]
```

```
stopCluster(cl)
registerDoSEQ()

best_xgb_params <- fit_xgb_tuned$bestTune
```

12.4 Tuning Results

```
tuning_results <- fit_xgb_tuned$results |>
  arrange(desc(ROC)) |>
  mutate(Rank = row_number()) |>
  select(Rank, `Mean CV AUC` = ROC, `SD CV AUC` = ROCSD,
         nrounds, max_depth, eta, gamma, colsample_bytree,
         min_child_weight, subsample)

tuning_results |>
  slice_head(n = 5) |>
  knitr::kable(digits = c(0, 4, 4, 0, 0, 3, 2, 2, 0, 2),
               caption = "Top 5 of 20 random XGBoost configurations (3-fold CV on the
        tuning sample)")
```

Top 5 of 20 random XGBoost configurations (3-fold CV on the tuning sample)

Rank	Mean CV AUC	SD CV AUC	nrounds	max_depth	eta	gamma	colsample_bytree	min_child_weight	subsample
1	0.7085	0.0213	300	2	0.047	4.00	0.95	10	0.86
2	0.7064	0.0268	300	2	0.028	3.12	0.69	1	0.90
3	0.7056	0.0257	200	4	0.021	0.74	0.81	8	0.94
4	0.6999	0.0169	450	2	0.060	4.36	0.67	8	0.74
5	0.6971	0.0266	350	8	0.022	1.81	0.62	1	0.82

```
best_tuning <- tuning_results |> slice(1)

# Independent check of the best configuration's fold AUCs with pROC.
best_fold_check <- fit_xgb_tuned$pred |>
  group_by(Resample) |>
  summarise(pROC_auc = as.numeric(auc(obs, Default, levels = c("NoDefault", "Default"),
                                      direction = "<", quiet = TRUE))) |>
  left_join(select(fit_xgb_tuned$resample, Resample, caret_auc = ROC), by = "Resample")

tibble(Item  = c("Configurations evaluated",
                 "Best mean CV AUC",
                 "Best SD CV AUC",
                 "AUC range across configurations",
                 "Total tuning runtime (min)",
                 "Max |pROC − caret| for best configuration"),
       Value = c(format(nrow(fit_xgb_tuned$results)),
                 sprintf("%.4f", best_tuning$`Mean CV AUC`),
                 sprintf("%.4f", best_tuning$`SD CV AUC`),
```

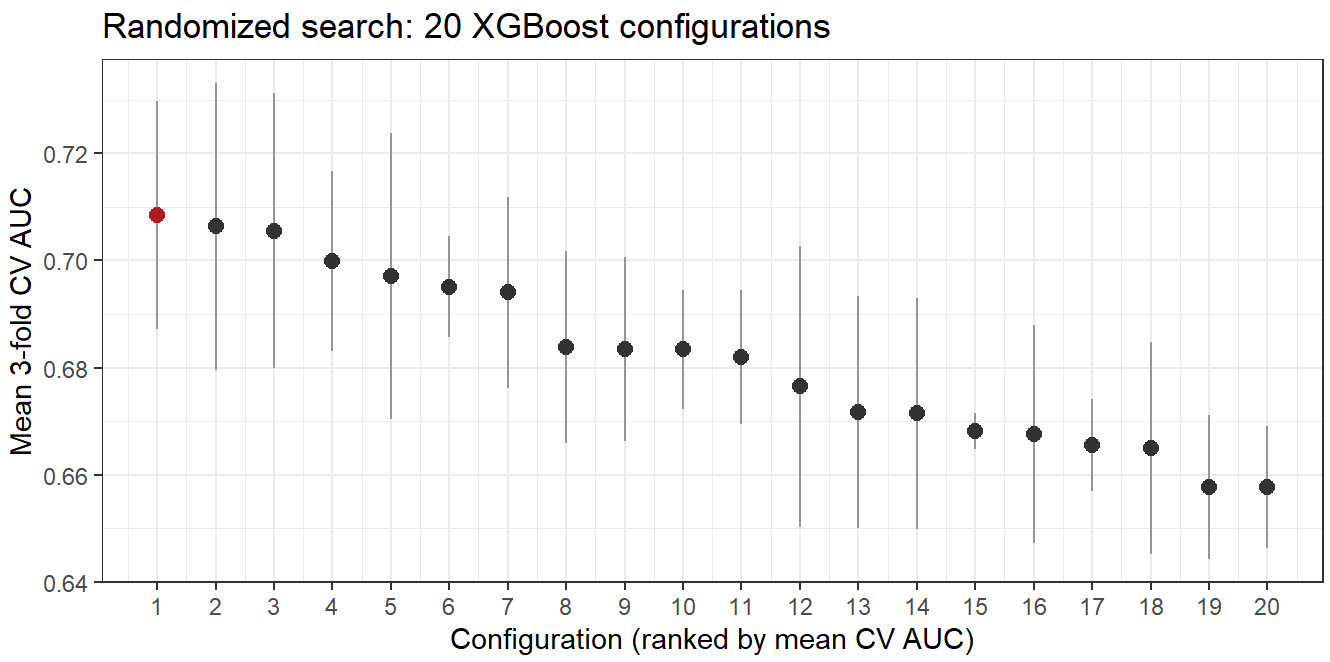
```
                sprintf("%.4f to %.4f", min(tuning_results$`Mean CV AUC`),
                        max(tuning_results$`Mean CV AUC`)),
                sprintf("%.2f", time_tuning / 60),
                format(max(abs(best_fold_check$pROC_auc - best_fold_check$caret_auc)))))
        |>
  knitr::kable(caption = "Randomized search summary")
```

Randomized search summary

Item	Value
Configurations evaluated	20
Best mean CV AUC	0.7085
Best SD CV AUC	0.0213
AUC range across configurations	0.6577 to 0.7085
Total tuning runtime (min)	0.19
Max |pROC - caret| for best configuration	0

The plot shows the mean CV AUC of all 20 configurations, ranked, with a bar of ±1 fold standard deviation around each.

```
tuning_results |>
  ggplot(aes(x = Rank, y = `Mean CV AUC`)) +
  geom_errorbar(aes(ymin = `Mean CV AUC` - `SD CV AUC`,
                    ymax = `Mean CV AUC` + `SD CV AUC`),
                width = 0, colour = "grey60") +
  geom_point(aes(colour = Rank == 1), size = 2.5, show.legend = FALSE) +
  scale_colour_manual(values = c(`TRUE` = "firebrick", `FALSE` = "grey20")) +
  scale_x_continuous(breaks = 1:n_configs) +
  labs(x = "Configuration (ranked by mean CV AUC)", y = "Mean 3-fold CV AUC",
       title = "Randomized search: 20 XGBoost configurations") +
  theme_bw(base_size = 11)
```



Mean 3-fold CV AUC of the 20 random configurations, ranked; bars show ±1 fold SD and the selected configuration is highlighted.

12.5 Tuning Interpretation

The highest mean CV AUC, **0.7085** (SD 0.0213 across folds), came from `nrounds =` 300, `max_depth =` 2, `eta =` 0.047, `gamma =` 4, `colsample_bytree =` 0.95, `min_child_weight =` 10, and `subsample =` 0.86. These values are stored in `best_xgb_params` for the next step.

The 20 configurations ranged from 0.6577 to 0.7085, a spread of 0.0508, so the settings matter. The top of the ranking is less decisive. The best configuration leads the second by only 0.0021, well inside the roughly 0.02 fold-to-fold standard deviation that comes with assessment folds of about 1,670 rows. The top few configurations are therefore roughly equivalent, and the winner is best read as one good setting, not a precisely optimal one.

The clearest pattern is the learning rate. The seven best configurations all use `eta` of 0.079 or lower, and the remaining thirteen all use 0.099 or higher. The selected model combines that slow learning rate with shallow trees (depth 2) and strong regularization: a high `gamma` and the largest `min_child_weight` in the range, both of which make the trees more reluctant to split. Small, cautious trees are a sensible fit for a training fold of about 3,300 rows with roughly 270 defaults. No consistent pattern appears for the other parameters among the top configurations; depth 8 also appears in the top five, for example.

Randomized search was used instead of an exhaustive grid because seven parameters with even a few values each would mean thousands of combinations. Random draws cover the whole range of every parameter with a fixed budget of 20 fits per fold, which works well when only a few parameters matter most; here, the learning rate appears to account for much of the difference. Tuning on about 5,000 rows keeps those 60 fits fast (0.19 minutes in total) while still giving every configuration a cross-validated, like-for-like comparison.

These AUCs should not be compared directly with the 0.7575 from the 50,001-row exploration sample. They come from a different, ten-times-smaller sample, where each model trains on far less data, so lower AUCs are expected for every configuration. The tuning CV result is used only to choose hyperparameters. Whether the tuned settings actually outperform the original configuration has not been established yet. That will be judged by fitting on the development data and evaluating once on the untouched holdout set. The holdout must stay out of tuning entirely: if it guided the choice of settings, its AUC would no longer be an honest estimate of performance on new applicants.

13 Final Internal Model Evaluation

Every modeling decision has now been made using development data only. The 20% holdout set was untouched throughout the candidate-model comparison, the imbalance experiment, and hyperparameter tuning, so it can now provide an honest, one-time estimate of out-of-sample performance. The selected model is:

- **XGBoost**,
- with **no class-imbalance adjustment**,
- using the **hyperparameters chosen by the randomized search** (`best_xgb_params`).

13.1 Development and Holdout Data

The model is trained on all development rows and evaluated on the holdout rows, using the same outcome coding and predictor set as every earlier step. `SK_ID_CURR` and `TARGET` are excluded from the predictors, and no weighting or resampling is applied. The Kaggle test file is not used here.

```
development_model_data <- development_data |>
  mutate(TARGET = factor(if_else(TARGET == 1, "Default", "NoDefault"),
                         levels = c("Default", "NoDefault"))) |>
  select(-SK_ID_CURR)

holdout_model_data <- holdout_data |>
  mutate(TARGET = factor(if_else(TARGET == 1, "Default", "NoDefault"),
                         levels = c("Default", "NoDefault"))) |>
  select(-SK_ID_CURR)

dev_predictors     <- setdiff(names(development_model_data), "TARGET")
holdout_predictors <- setdiff(names(holdout_model_data), "TARGET")

tibble(Item  = c("Development rows",
                 "Holdout rows",
                 "Development predictors",
                 "Holdout predictors",
                 "Predictor names align",
                 "Same predictors as earlier modeling",
                 "SK_ID_CURR among predictors"),
       Value = c(format(nrow(development_model_data), big.mark = ","),
                 format(nrow(holdout_model_data), big.mark = ","),
```

```
                format(length(dev_predictors)),
                format(length(holdout_predictors)),
                format(identical(dev_predictors, holdout_predictors)),
                format(identical(names(development_model_data), names(model_data))),
                format("SK_ID_CURR" %in% c(dev_predictors, holdout_predictors)))) |>
  knitr::kable(caption = "Final evaluation data")
```

Final evaluation data

Item	Value
Development rows	246,009
Holdout rows	61,502
Development predictors	167
Holdout predictors	167
Predictor names align	TRUE
Same predictors as earlier modeling	TRUE
SK_ID_CURR among predictors	FALSE

13.2 Fitting the Selected Model

The model is fit once on the complete development set. `trainControl(method = "none")` tells caret to skip resampling and tuning entirely; it simply fits the single configuration in `best_xgb_params` with the same custom XGBoost method used throughout. With only one model to fit, XGBoost uses 4 threads directly instead of a cluster of workers.

```
tibble(Parameter = names(best_xgb_params),
       Value     = unlist(best_xgb_params)) |>
  knitr::kable(caption = "Selected hyperparameters (best_xgb_params)")
```

Selected hyperparameters (best_xgb_params)

Parameter	Value
nrounds	300.000
max_depth	2.000
eta	0.047
gamma	4.000
colsample_bytree	0.950
min_child_weight	10.000
subsample	0.860

```
set.seed(6493)
time_final_internal <- system.time(
  final_internal_xgb <- train(
    TARGET ~ ., data = development_model_data,
    method    = xgb_tree_model,
    trControl = trainControl(method = "none", classProbs = TRUE),
```

```
    tuneGrid  = best_xgb_params,
    nthread   = 4
  )
)["elapsed"]
```

13.3 Development and Holdout Performance

The first AUC below is **in-sample**: the model is scored on the same development rows it was trained on, which usually flatters it. The second is the holdout AUC, the primary final internal estimate. Both use the predicted probability of `Default`.

```
dev_prob     <- predict(final_internal_xgb, newdata = development_model_data, type =
        "prob")$Default
holdout_prob <- predict(final_internal_xgb, newdata = holdout_model_data,     type =
        "prob")$Default

development_auc <- as.numeric(auc(development_model_data$TARGET, dev_prob,
                                  levels = c("NoDefault", "Default"),
                                  direction = "<", quiet = TRUE))
holdout_auc     <- as.numeric(auc(holdout_model_data$TARGET, holdout_prob,
                                  levels = c("NoDefault", "Default"),
                                  direction = "<", quiet = TRUE))
auc_gap <- development_auc - holdout_auc

tibble(Dataset   = c("Development", "Holdout"),
       Rows      = c(nrow(development_model_data), nrow(holdout_model_data)),
       `ROC AUC` = c(development_auc, holdout_auc),
       Notes     = c("In-sample / training", "Untouched out-of-sample evaluation")) |>
  knitr::kable(digits = 4, format.args = list(big.mark = ","),
               caption = "Selected XGBoost model: development vs holdout ROC AUC")
```

Selected XGBoost model: development vs holdout ROC AUC

Dataset	Rows	ROC AUC	Notes
Development	246,009	0.7639	In-sample / training
Holdout	61,502	0.7556	Untouched out-of-sample evaluation

```
tibble(Item  = c("Training runtime (min)", "Development AUC", "Holdout AUC",
                 "AUC gap (development - holdout)"),
       Value = c(sprintf("%.2f", time_final_internal / 60), fmt(development_auc),
                 fmt(holdout_auc), fmt(auc_gap))) |>
  knitr::kable(caption = "Final internal evaluation summary")
```

Final internal evaluation summary

Item	Value
Training runtime (min)	0.24
Development AUC	0.7639

Item	Value
Holdout AUC	0.7556
AUC gap (development - holdout)	0.0083

The table below gives threshold-based diagnostics on the holdout set at the default 0.50 cutoff. They are **secondary and descriptive only**: 0.50 was not optimized, no threshold is chosen using the holdout, and ROC AUC remains the main evaluation metric.

```
holdout_class <- if_else(holdout_prob >= 0.5, "Default", "NoDefault")
holdout_obs   <- as.character(holdout_model_data$TARGET)

holdout_diagnostics <- tibble(
  Sensitivity = mean(holdout_class[holdout_obs == "Default"] == "Default"),
  Specificity = mean(holdout_class[holdout_obs == "NoDefault"] == "NoDefault"),
  Accuracy    = mean(holdout_class == holdout_obs)
)

holdout_diagnostics |>
  knitr::kable(digits = 4, caption = "Holdout diagnostics at a 0.50 threshold
        (secondary)")
```

Holdout diagnostics at a 0.50 threshold (secondary)

Sensitivity	Specificity	Accuracy
0.005	0.9998	0.9195

13.4 Interpretation of the Final Internal Evaluation

On 61,502 holdout applicants the model never saw, the selected XGBoost model reaches a **ROC AUC of 0.7556**. If one applicant who went on to have payment difficulties and one who did not are picked at random, the model gives the first a higher predicted risk about 76% of the time, against 50% for the majority-class benchmark.

The in-sample development AUC is 0.7639, only 0.0083 higher. Some gap is expected, because the model is being scored on the rows it learned from. A gap this small suggests no meaningful overfitting, which fits the shallow, heavily regularized trees selected by tuning. The holdout AUC is the more informative number: training AUC rewards a model for fitting its own data, while the holdout AUC measures what matters for Home Credit, ranking new applicants it has not seen.

For context only, the fixed-parameter XGBoost model in the initial comparison scored a mean CV AUC of 0.7575, and the best tuning configuration scored 0.7085. Neither is comparable with the holdout result. The first came from different parameters on a 50,001-row sample with different folds, and the second from 3,300-row training folds within a 5,000-row sample. Those numbers served to choose a model. The holdout AUC, from a model trained on all development data and scored on untouched rows, is now the strongest internal estimate of how well the model generalizes.

The model must not be changed in response to this result. Once the holdout influences a decision, such as revisiting the parameters, the features, or the imbalance strategy, it becomes another round of model

selection, and its AUC stops being an honest out-of-sample estimate.

At the untuned 0.50 threshold the model flags almost no applicants as defaults (sensitivity 0.0050), and its accuracy of 91.95% is essentially the 91.93% majority-class rate in the holdout. As with the benchmark, this reflects the rarity of defaults and the fact that an unadjusted model's probabilities rarely exceed 0.50. It says nothing about ranking quality, which is why AUC remains the evaluation metric. Any operating threshold would have to be chosen from development data for a specific business purpose.

13.5 Feature Importance

The importance scores below come from the fitted model and are descriptive only; they do not change it. Gain measures how much each predictor's splits improved the model's fit, summed over all trees, as a share of the total. Because caret dummy-encodes factors, a categorical predictor appears as one column per level (for example, `NAME_EDUCATION_TYPEHigher education`), so the source variable is shown alongside each column.

```
# Map each dummy-encoded column back to the prepared variable it came from.
encoding_terms <- terms(TARGET ~ ., data = development_model_data)
encoded_matrix <- model.matrix(encoding_terms, development_model_data[1:2, ])
column_source  <- tibble(
  Feature           = colnames(encoded_matrix),
  `Source variable` = c("(Intercept)", attr(encoding_terms, "term.labels"))
        [attr(encoded_matrix, "assign") + 1]
)

feature_importance <- xgb.importance(model = final_internal_xgb$finalModel$booster) |>
  as_tibble() |>
  left_join(column_source, by = "Feature") |>
  arrange(desc(Gain))

top_importance <- feature_importance |>
  slice_head(n = 15) |>
  select(Predictor = Feature, `Source variable`, Gain)

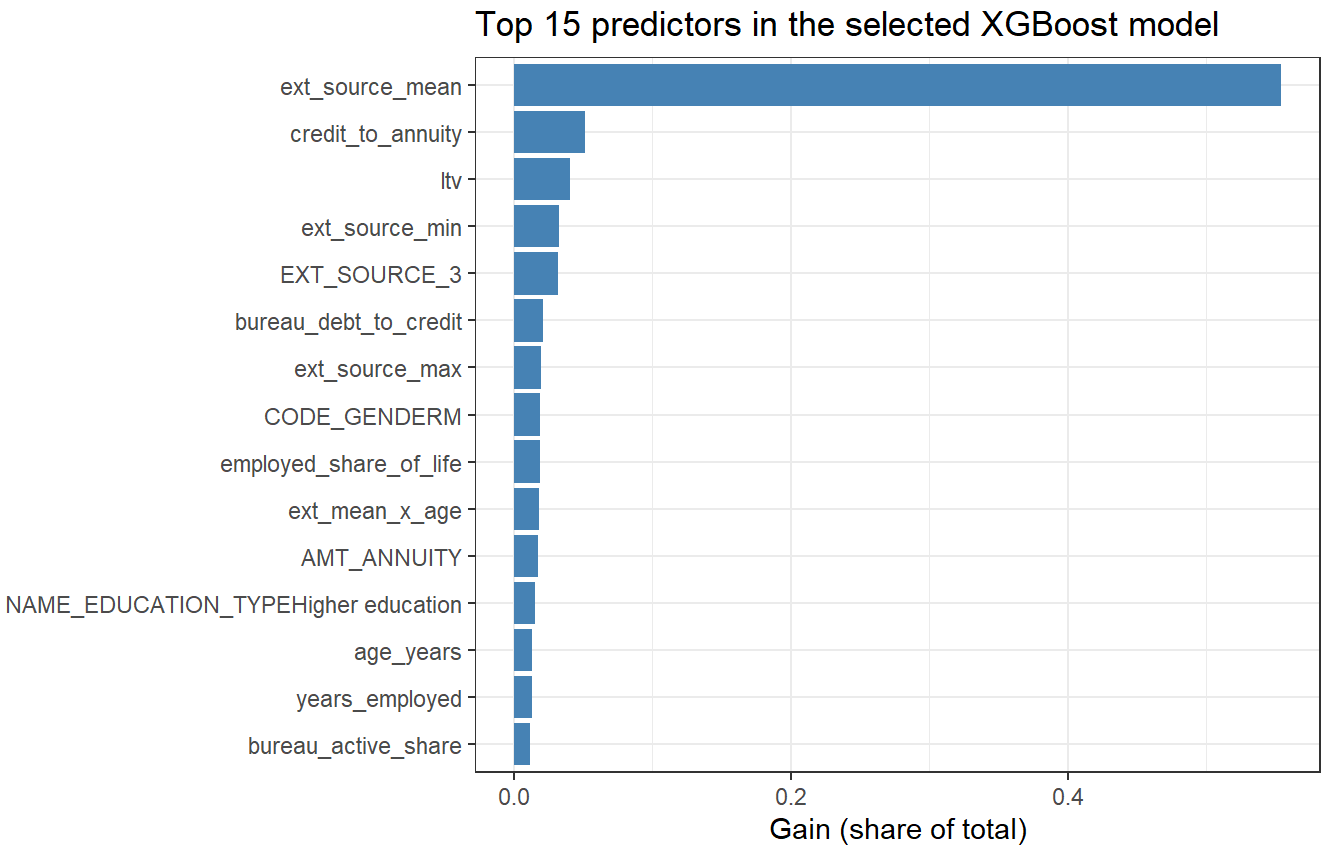
top_importance |>
  knitr::kable(digits = 4, caption = "Top 15 predictors by gain")
```

Top 15 predictors by gain

Predictor	Source variable	Gain
ext_source_mean	ext_source_mean	0.5543
credit_to_annuity	credit_to_annuity	0.0514
ltv	ltv	0.0408
ext_source_min	ext_source_min	0.0325
EXT_SOURCE_3	EXT_SOURCE_3	0.0317
bureau_debt_to_credit	bureau_debt_to_credit	0.0209
ext_source_max	ext_source_max	0.0194

Predictor	Source variable	Gain
CODE_GENDERM	CODE_GENDER	0.0192
employed_share_of_life	employed_share_of_life	0.0187
ext_mean_x_age	ext_mean_x_age	0.0185
AMT_ANNUITY	AMT_ANNUITY	0.0175
NAME_EDUCATION_TYPEHigher education	NAME_EDUCATION_TYPE	0.0153
age_years	age_years	0.0132
years_employed	years_employed	0.0130
bureau_active_share	bureau_active_share	0.0116

```
top_importance |>
  ggplot(aes(x = Gain, y = fct_reorder(Predictor, Gain))) +
  geom_col(fill = "steelblue") +
  labs(x = "Gain (share of total)", y = NULL,
       title = "Top 15 predictors in the selected XGBoost model") +
  theme_bw(base_size = 11)
```



Top 15 predictors by gain in the XGBoost model fit on the development data.

The external credit scores dominate. `ext_source_mean` alone accounts for 55% of total gain, and every predictor built from the external scores combined (the raw scores, their mean, minimum and maximum, the interaction with age, and their missing-value flags) accounts for 67%. This matches the EDA, which found the external scores to be the strongest single predictors. The rest of the top 15 falls into a few broad groups:

- **Loan structure and affordability:** `credit_to_annuity` (a proxy for loan term), loan-to-value (`ltv`), and the annuity amount.
- **Credit bureau history:** the share of bureau debt still outstanding and the share of credits still active.
- **Stability and demographics:** age, years employed, the share of life spent employed, education, and gender.

Several of these are features engineered in `data_preparation.R` from EDA findings (`ltv`, `credit_to_annuity`, `employed_share_of_life`, the external-score summaries, and the bureau aggregates). The model used only 70 of the 300 encoded columns at all, which is consistent with its shallow, regularized trees.

Importance shows which variables the model relied on to make its predictions. It does not show that these variables cause default, and it does not say in which direction each one moves risk. The appearance of gender and age among the top predictors also deserves attention: both are protected characteristics in many lending contexts, so their use would need a fairness and regulatory review before any deployment, as the EDA recommended for age.

14 Final Model and Kaggle Submission

The selected XGBoost specification was first evaluated on the untouched internal holdout (AUC 0.7556). With that evaluation complete, the same specification is now refit on **all 307,511 labeled applicants**, so the final model learns from every labeled row, including the former holdout. This full-data fit is used only to produce Kaggle predictions. Nothing about the model changes: same features, same hyperparameters, no imbalance adjustment, and no tuning. The Kaggle test set is used only for prediction.

14.1 Full Training and Kaggle Test Data

```
full_model_data <- train_prepared |>
  mutate(TARGET = factor(if_else(TARGET == 1, "Default", "NoDefault"),
                         levels = c("Default", "NoDefault"))) |>
  select(-SK_ID_CURR)

kaggle_test_x <- test_prepared |> select(-SK_ID_CURR)

full_predictors <- setdiff(names(full_model_data), "TARGET")
factor_levels   <- function(df) map(select(df, where(is.factor)), levels)

kaggle_data_checks <- tibble(
  Check  = c("Full training rows",
             "Kaggle test rows",
             "Predictors (before dummy encoding)",
             "Train/test predictor names match",
             "Factor levels identical in train and test",
             "Missing values (train, test)",
```

```
            "SK_ID_CURR among predictors"),
  Result = c(format(nrow(full_model_data), big.mark = ","),
             format(nrow(kaggle_test_x), big.mark = ","),
             format(length(full_predictors)),
             format(identical(full_predictors, names(kaggle_test_x))),
             format(identical(factor_levels(full_model_data[full_predictors]),
                              factor_levels(kaggle_test_x))),
             paste(sum(is.na(full_model_data)), sum(is.na(kaggle_test_x)), sep = ", "),
             format("SK_ID_CURR" %in% c(full_predictors, names(kaggle_test_x))))
)
stopifnot(nrow(full_model_data) == 307511, nrow(kaggle_test_x) == 48744,
          identical(full_predictors, names(kaggle_test_x)))

kaggle_data_checks |> knitr::kable(caption = "Final training and Kaggle test data")
```

Final training and Kaggle test data

Check	Result
Full training rows	307,511
Kaggle test rows	48,744
Predictors (before dummy encoding)	167
Train/test predictor names match	TRUE
Factor levels identical in train and test	TRUE
Missing values (train, test)	0, 0
SK_ID_CURR among predictors	FALSE

14.2 Fitting the Final Model

As in the internal evaluation, `trainControl(method = "none")` fits the single selected configuration once, with no resampling or tuning. The fitted model is saved to the git-ignored `models/` folder at the repository root.

```
set.seed(6493)
time_final_full <- system.time(
  final_full_xgb <- train(
    TARGET ~ ., data = full_model_data,
    method    = xgb_tree_model,
    trControl = trainControl(method = "none", classProbs = TRUE),
    tuneGrid  = best_xgb_params,
    nthread   = 4
  )
)["elapsed"]

model_path <- "../models/final_home_credit_xgb.rds"
dir.create(dirname(model_path), showWarnings = FALSE)
saveRDS(final_full_xgb, model_path)

tibble(Parameter = c(names(best_xgb_params), "Imbalance adjustment",
```

```
                  "Training rows", "Training runtime (min)"),
       Value     = c(map_chr(best_xgb_params, format), "None",
                     format(nrow(full_model_data), big.mark = ","),
                     sprintf("%.2f", time_final_full / 60))) |>
  knitr::kable(caption = "Final full-data XGBoost model")
```

Final full-data XGBoost model

Parameter	Value
nrounds	300
max_depth	2
eta	0.047
gamma	4
colsample_bytree	0.95
min_child_weight	10
subsample	0.86
Imbalance adjustment	None
Training rows	307,511
Training runtime (min)	0.30

14.3 Kaggle Predictions and Submission

The model predicts the probability of `TARGET = 1` (Default) for every Kaggle test applicant. The submission keeps probabilities, not class labels. `sample_submission.csv` provides the required structure and row order. Predictions are matched to it by `SK_ID_CURR` rather than by position.

```
kaggle_prob <- predict(final_full_xgb, newdata = kaggle_test_x, type = "prob")$Default

predictions <- tibble(SK_ID_CURR = test_prepared$SK_ID_CURR, TARGET = kaggle_prob)
stopifnot(nrow(predictions) == 48744, !anyNA(kaggle_prob),
          all(kaggle_prob >= 0 & kaggle_prob <= 1))

sample_submission <- read_csv("data/sample_submission.csv", show_col_types = FALSE)
stopifnot(identical(names(sample_submission), c("SK_ID_CURR", "TARGET")),
          !anyDuplicated(sample_submission$SK_ID_CURR),
          setequal(sample_submission$SK_ID_CURR, test_prepared$SK_ID_CURR))

submission <- sample_submission |>
  select(SK_ID_CURR) |>
  left_join(predictions, by = "SK_ID_CURR")

submission_path <- "../submissions/home_credit_xgboost_submission.csv"
dir.create(dirname(submission_path), showWarnings = FALSE)
# write_csv never writes row names or an index column.
write_csv(submission, submission_path)

# Validate the file as written, not just the in-memory table.
```

```r
written <- read_csv(submission_path, show_col_types = FALSE)
submission_checks <- tibble(
  Check  = c("Row count = 48,744",
             "Column count = 2",
             "Columns are exactly SK_ID_CURR, TARGET",
             "No duplicate SK_ID_CURR",
             "No missing TARGET",
             "TARGET minimum >= 0",
             "TARGET maximum <= 1",
             "IDs match the Kaggle test file",
             "Order matches sample_submission.csv"),
  Passed = c(nrow(written) == 48744,
             ncol(written) == 2,
             identical(names(written), c("SK_ID_CURR", "TARGET")),
             !anyDuplicated(written$SK_ID_CURR),
             !anyNA(written$TARGET),
             min(written$TARGET) >= 0,
             max(written$TARGET) <= 1,
             setequal(written$SK_ID_CURR, test_prepared$SK_ID_CURR),
             identical(written$SK_ID_CURR, sample_submission$SK_ID_CURR))
)
stopifnot(all(submission_checks$Passed))

submission_checks |> knitr::kable(caption = "Submission file checks")
```

Submission file checks

Check	Passed
Row count = 48,744	TRUE
Column count = 2	TRUE
Columns are exactly SK_ID_CURR, TARGET	TRUE
No duplicate SK_ID_CURR	TRUE
No missing TARGET	TRUE
TARGET minimum >= 0	TRUE
TARGET maximum <= 1	TRUE
IDs match the Kaggle test file	TRUE
Order matches sample_submission.csv	TRUE

```r
tibble(Item  = c("Submission file", "Dimensions", "Minimum", "Maximum",
                 "Mean", "Median", "Missing values", "Duplicate SK_ID_CURR"),
       Value = c(sub("^\\.\\./", "", submission_path),
                 paste(nrow(written), "rows x", ncol(written), "columns"),
                 sprintf("%.4f", min(written$TARGET)),
                 sprintf("%.4f", max(written$TARGET)),
                 sprintf("%.4f", mean(written$TARGET)),
                 sprintf("%.4f", median(written$TARGET)),
                 format(sum(is.na(written))),
                 format(sum(duplicated(written$SK_ID_CURR))))) |>
  knitr::kable(caption = "Predicted default probabilities for Kaggle applicants")
```

Predicted default probabilities for Kaggle applicants

Item	Value
Submission file	submissions/home_credit_xgboost_submission.csv
Dimensions	48744 rows x 2 columns
Minimum	0.0063
Maximum	0.6252
Mean	0.0831
Median	0.0565
Missing values	0
Duplicate SK_ID_CURR	0

```
written |>
  slice_head(n = 5) |>
  knitr::kable(digits = 4, caption = "First 5 submission rows")
```

First 5 submission rows

SK_ID_CURR	TARGET
100001	0.0415
100005	0.1392
100013	0.0188
100028	0.0426
100038	0.1627

The submission contains one predicted default probability for each of the 48,744 Kaggle applicants, in the order of `sample_submission.csv`. The mean predicted probability is 8.31%. For context only, the observed default rate in the training data is 8.0729%. The two need not match: the Kaggle applicants are a separate set of applicants, and Kaggle scores the submission on ranking (ROC AUC), not on the average probability.

14.4 Kaggle Results

The submission was made after the competition deadline, as a late submission, because the original competition has ended. Kaggle still scores late submissions, but they do not appear on the leaderboard. Kaggle reports two scores. The **public** score is computed on one portion of the test applicants, and the **private** score on the remaining portion; the private score determined the official final ranking. These scores were reported by Kaggle and are entered here as recorded.

```
kaggle_public_auc  <- 0.75749
kaggle_private_auc <- 0.75276

tibble(Evaluation = c("Internal holdout", "Kaggle public", "Kaggle private"),
       `ROC AUC`  = c(sprintf("%.4f", holdout_auc),
                      sprintf("%.5f", kaggle_public_auc),
                      sprintf("%.5f", kaggle_private_auc)),
```

```
      `Model scored` = c("Fit on development data (246,009 rows)",
                         "Fit on all labeled data (307,511 rows)",
                         "Fit on all labeled data (307,511 rows)"),
      `Applicants scored` = c("61,502 labeled holdout applicants",
                              "Public portion of Kaggle test set",
                              "Private portion of Kaggle test set")) |>
  knitr::kable(caption = "Final ROC AUC: internal holdout and Kaggle")
```

Final ROC AUC: internal holdout and Kaggle

Evaluation	ROC AUC	Model scored	Applicants scored
Internal holdout	0.7556	Fit on development data (246,009 rows)	61,502 labeled holdout applicants
Kaggle public	0.75749	Fit on all labeled data (307,511 rows)	Public portion of Kaggle test set
Kaggle private	0.75276	Fit on all labeled data (307,511 rows)	Private portion of Kaggle test set

Public Kaggle ROC AUC: 0.75749. Private Kaggle ROC AUC: 0.75276.

Both Kaggle scores are close to the internal holdout AUC of 0.7556. The public score is 0.0019 higher and the private score is 0.0029 lower. That agreement is evidence that the internal holdout gave a reasonably representative estimate of how the model performs on unseen applicants. The comparison is not exact. The Kaggle predictions came from the model refit on all labeled data rather than the development-only model, the three scores come from different applicants, and nothing here shows that the Kaggle test applicants share the holdout's distribution. Even so, differences of a few thousandths of AUC are small, and none of the three scores suggests that the model overfit its training data.

15 Modeling Summary

The table below collects the main results. They come from **different evaluation settings**, so they are not all one like-for-like comparison. Candidate-model and imbalance results are mean 3-fold CV AUCs on the 50,001-row exploration sample, the tuning result is a CV AUC on a 5,000-row sample, and the final results come from the holdout set and Kaggle's test set.

```
tibble(
  Stage = c("Benchmark", "Candidate models", "Candidate models", "Candidate models",
            "Imbalance (XGBoost)", "Imbalance (XGBoost)",
            "Tuning", "Final internal", "Final internal", "Kaggle", "Kaggle"),
  `Model / experiment` = c("Majority class", "Ridge logistic regression", "Random
        forest",
                           "XGBoost (fixed parameters, unadjusted)",
                           "Positive-class weighting", "Random downsampling",
                           "Best of 20 random configurations",
                           "Selected XGBoost: development (in-sample)",
                           "Selected XGBoost: untouched holdout",
                           "Selected XGBoost: public score", "Selected XGBoost: private
        score"),
  Evaluation = c("Training data", rep("3-fold CV, exploration sample", 5),
                 "3-fold CV, 5,000-row tuning sample", "Training rows",
                 "61,502 holdout rows", "Kaggle test set", "Kaggle test set"),
```

```
  `ROC AUC` = c(sprintf("%.4f", benchmark_auc),
                fmt(lr$"Mean CV AUC"), fmt(rf$"Mean CV AUC"), fmt(xgb$"Mean CV AUC"),
                fmt(imb_wt$"Mean CV AUC"), fmt(imb_down$"Mean CV AUC"),
                fmt(best_tuning$"Mean CV AUC"),
                fmt(development_auc), fmt(holdout_auc),
                sprintf("%.5f", kaggle_public_auc), sprintf("%.5f", kaggle_private_auc))
) |>
  knitr::kable(caption = "Summary of modeling results (ROC AUC)")
```

Summary of modeling results (ROC AUC)

Stage	Model / experiment	Evaluation	ROC AUC
Benchmark	Majority class	Training data	0.5000
Candidate models	Ridge logistic regression	3-fold CV, exploration sample	0.7473
Candidate models	Random forest	3-fold CV, exploration sample	0.7262
Candidate models	XGBoost (fixed parameters, unadjusted)	3-fold CV, exploration sample	0.7575
Imbalance (XGBoost)	Positive-class weighting	3-fold CV, exploration sample	0.7514
Imbalance (XGBoost)	Random downsampling	3-fold CV, exploration sample	0.7495
Tuning	Best of 20 random configurations	3-fold CV, 5,000-row tuning sample	0.7085
Final internal	Selected XGBoost: development (in-sample)	Training rows	0.7639
Final internal	Selected XGBoost: untouched holdout	61,502 holdout rows	0.7556
Kaggle	Selected XGBoost: public score	Kaggle test set	0.75749
Kaggle	Selected XGBoost: private score	Kaggle test set	0.75276

- **Benchmark.** Predicting “no default” for everyone gives 0.919271 accuracy but a ROC AUC of 0.5000. Accuracy is misleading at an 8.07% default rate, so ROC AUC was the selection metric throughout.
- **Candidate models.** On identical stratified 3-fold CV folds, XGBoost (0.7575) beat ridge logistic regression (0.7473) and random forest (0.7262). It led in every fold and was also the fastest to fit.
- **Class imbalance.** Positive-class weighting (0.7514) and random downsampling (0.7495) both scored below unadjusted XGBoost (0.7575). Both raised sensitivity at a 0.50 threshold but reduced ranking performance, so the unadjusted model was retained.
- **Tuning.** A randomized search scored 20 unique XGBoost configurations with stratified 3-fold CV on a 5,000-row development sample, optimizing ROC AUC. It selected `nrounds =` 300, `max_depth =` 2, `eta =` 0.047, `gamma =` 4, `colsample_bytree =` 0.95, `min_child_weight =` 10, and `subsample =` 0.86. Its CV AUC of 0.7085 came from a much smaller sample than the earlier 0.7575 and does not indicate that tuning made the model worse.
- **Final internal evaluation.** Fit on all development data, the selected model scored 0.7639 in-sample and 0.7556 on the untouched holdout, a gap of 0.0083.
- **Kaggle.** Refit on all 307,511 labeled rows, the model scored 0.75749 public and 0.75276 private.

XGBoost was selected because it had the highest cross-validated ROC AUC among the candidates, consistently across folds, at the lowest runtime, and no imbalance adjustment improved on it. The small development-to-holdout gap and the close agreement between holdout and Kaggle scores show limited evidence of overfitting.

16 Findings and Business Interpretation

The business question was whether Home Credit could better estimate which applicants are likely to have payment difficulties, so that it can manage risk while still lending to applicants who can repay. The model answers that question: it assigns each applicant a predicted probability of payment difficulty, which ranks applicants from highest to lowest estimated risk. Its holdout AUC of 0.7556, confirmed by Kaggle scores of about 0.75 to 0.76, means it ranks applicants who later had payment difficulties above those who did not much more often than chance (0.50). A higher AUC means better ranking, and better ranking makes these uses possible:

- **Prioritizing applications for additional review**, by routing the highest-risk applications to an underwriter first.
- **Focusing limited underwriting resources** on the applications where review is most likely to matter.
- **Identifying higher-risk applicants for additional verification** of income, employment, or documents.
- **Supporting risk-management decisions**, such as loan amount or terms, alongside existing policy.

The model should support lending review and policy, not replace them. Its scores should not be used to automatically deny applicants.

What drives the predictions:

- **External scores.** Features derived from the external credit scores were by far the most influential, at about 67% of total gain. Loan structure and affordability, bureau history, and employment stability also contributed.
- **Predictive, not causal.** Feature importance shows which variables the model relied on to predict default, not which variables cause it.
- **Protected characteristics.** Age and gender appeared among the influential predictors. Both are protected characteristics in many lending contexts, so a fairness review and a legal and compliance review would be needed before any real-world use.

This project includes no cost-benefit analysis, so no financial savings are claimed. Choosing how to act on the scores, including any decision threshold, would require Home Credit's own costs, policies, and risk appetite.

17 Limitations

These limitations qualify the results without invalidating them:

1. **Target information before CV.** Some EDA-derived feature choices, such as the fixed bands and the selection of missing-value indicator columns, used `TARGET` information from the full training data before cross-validation. CV estimates may therefore be slightly optimistic. The same applies, to a lesser degree, to the holdout, whose rows were part of the training data used in preparation. The Kaggle test outcomes were never used anywhere, and the Kaggle scores are close to the internal estimates, which suggests the effect is small.

2. **Preprocessing learned before CV.** Most preprocessing parameters, such as imputation medians, factor levels, the income cap, and cut points, were learned on the full training data before CV, although they did not use `TARGET`.
3. **Supplementary data.** Only the bureau table was incorporated. The other Home Credit tables (`previous_application`, `installments_payments`, `POS_CASH_balance`, `credit_card_balance`, and `bureau_balance`) were not used and may contain additional signal.
4. **Small tuning sample.** The 5,000-row tuning sample had 404 defaults, about 135 per assessment fold, so tuning AUCs were noisy (fold SD around 0.02), and the top configurations could not be clearly separated.
5. **No decision threshold.** The model was evaluated on discrimination (ROC AUC), not on an optimized lending threshold or a financial cost-benefit analysis.
6. **Fairness and compliance.** Fairness, bias, and regulatory compliance were not analyzed. That analysis would be required before deployment, particularly given the influence of age and gender.